Regression — Project Recipe Card

End-to-end recipe · model toolbox · metrics · CV · tuning · diagnostics · deployment. Deep companion: `regression_cheatsheet.md` + `regression.ipynb`

◆ Project recipe (11 steps)

1. **Frame** — target, units, success metric, decision the model serves
2. **Split FIRST** — train/val/test BEFORE any look at the data (avoid leakage)
3. **EDA** — target dist, missing, outliers, leakage candidates, target/feature correlations
4. **Feature pipeline** — `ColumnTransformer` → impute → encode → scale
5. **Baseline** — DummyRegressor (mean) + OLS; everything must beat it
6. **Model toolbox** — start linear, add Ridge/Lasso, then GBM
7. **CV** — KFold(5) or TimeSeriesSplit if temporal; nested for unbiased tuning
8. **Tune** — Optuna TPE (or GridSearchCV for ≤3 hp)
9. **Diagnostics** — residuals vs ŷ / feature / time; Q-Q plot
10. **Test ONCE** — final score on held-out test set
11. **Deploy** — pickle pipeline + log metrics + drift monitor

Universal skeleton: `Pipeline([("prep", ColumnTransformer(...)), ("model", Estimator())])` → fit on (X_tr, y_tr), grid-search, evaluate.

◆ Starter notebook skeleton

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split, KFold,
GridSearchCV
from sklearn.pipeline       import Pipeline
from sklearn.compose        import ColumnTransformer
from sklearn.impute         import SimpleImputer
from sklearn.preprocessing  import StandardScaler, OneHotEncoder
from sklearn.linear_model   import Ridge
from sklearn.metrics        import mean_squared_error, r2_score
import matplotlib.pyplot as plt
np.random.seed(42)

# 1. Split FIRST (3-way: train / val / test) — DO NOT touch test
until step 4
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)
X_tr, X_va, y_tr, y_va = train_test_split(X_tr, y_tr,
test_size=0.2, random_state=42)
# 60/20/20 final split; val used for early stopping + threshold-
style decisions

# 2. Build pipeline
num = ["age", "income"];  cat = ["region", "segment"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc",  StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
])
pipe = Pipeline([("prep", prep), ("model", Ridge())])

# 3. CV + grid search
grid = {"model__alpha": [0.01, 0.1, 1, 10, 100]}
gs = GridSearchCV(pipe, grid, cv=KFold(5, shuffle=True,
random_state=42),
                  scoring="neg_root_mean_squared_error",
n_jobs=-1)
gs.fit(X_tr, y_tr)

# 4. Test ONCE
y_hat = gs.predict(X_te)
print(np.sqrt(mean_squared_error(y_te, y_hat)), r2_score(y_te,
y_hat))
```

● Frame + EDA

Frame checklist

- **Target**: y_pred unit, range, what business decision uses it
- **Train domain**: which rows will the model see in prod?
- **Loss → metric**: symmetric (RMSE) or asymmetric (Pinball)?
- **Latency**: ms (linear) / s (GBM) / minutes (GP)?
- **Causal vs predictive**: do you need coefficients to mean something?

EDA snippets

```
df.info(); df.describe(include="all")
df.isna().sum().sort_values(ascending=False)
df[target].hist(bins=50); df[target].skew()
df.corr(numeric_only=True)[target].sort_values()
# pairplot for small dims:
import seaborn as sns; sns.pairplot(df.sample(500))
```

Target audit

- **Skew > 1** → try `np.log1p(y)`; inverse via `np.expm1`
- **Heavy tail** → quantile loss (GBM `objective="quantile"`) or Huber
- **Bounded (0,1)** → logit transform or Beta regression
- **Counts** → Poisson / NegBin GLM (not OLS)

```
# Canonical fix for log-target (no manual inverse needed):
from sklearn.compose import TransformedTargetRegressor
ttr = TransformedTargetRegressor(regressor=pipe,
                                 func=np.log1p,
inverse_func=np.expm1)
ttr.fit(X_tr, y_tr); y_hat = ttr.predict(X_te)   # already in
original units
```

Leakage candidates

Drop / quarantine before split: any feature derived from target, post-event features, future timestamps, ID columns, perfectly correlated proxies (e.g., `revenue` predicting `profit`).

● Feature pipeline (sklearn ≥1.5)

ColumnTransformer (canonical)

```
prep = ColumnTransformer(
    transformers=[
        ("num", num_pipe, num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore",
                              sparse_output=False), cat_cols),
        ("txt", TfidfVectorizer(max_features=5000), "text_col"),
    ],
    remainder="drop",         # drop unspecified
    verbose_feature_names_out=False,
).set_output(transform="pandas")
```

Use `set_output("pandas")` for dbg-friendly column names.

Imputation patterns

- **Median** numeric (robust to outliers)
- **Most frequent** categorical
- **Missing-as-category** for cat ('UNK')
- **KNNImputer** when correlated features predict missing
- **IterativeImputer** for MICE (slow, multivariate)

Scaling

Scaler	When
StandardScaler	linear, SVR, NN (default)
RobustScaler	outliers present
MinMaxScaler	NN with sigmoid, image data
None	tree-based models (invariant)

Feature engineering

```
# Date features (no leakage — current row only):
df["dow"] = df["dt"].dt.dayofweek
df["month"] = df["dt"].dt.month
# Interactions: PolynomialFeatures(degree=2,
interaction_only=True)
# Target encoding inside CV via category_encoders.TargetEncoder
```

▲ Model toolbox (climb the ladder)

1 · Dummy baseline

```
from sklearn.dummy import DummyRegressor
DummyRegressor(strategy="mean").fit(X_tr, y_tr).score(X_te,
y_te)
```

If your model can't beat this, something is wrong.

2 · OLS (LinearRegression)

```
from sklearn.linear_model import LinearRegression
LinearRegression().fit(X_tr, y_tr)
```

Interpretable; no hyperparams; can overfit with many features.

3 · Ridge / Lasso / ElasticNet

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
Ridge(alpha=1.0).fit(X_tr, y_tr)          # L2 — keep all, shrink
coefs
Lasso(alpha=0.1).fit(X_tr, y_tr)          # L1 — feature
selection
ElasticNet(alpha=0.1, l1_ratio=0.5)       # blend
```

α via CV (RidgeCV/LassoCV); **scale features first**.

4 · GLM family

```
from sklearn.linear_model import PoissonRegressor,
GammaRegressor, TweedieRegressor
PoissonRegressor(alpha=0.5).fit(X_tr, y_tr)   # counts / non-neg
GammaRegressor().fit(X_tr, y_tr)              # positive, right-
skew
TweedieRegressor(power=1.5).fit(X_tr, y_tr)   # zero-inflated
continuous
```

5 · Tree-based: RandomForest

```
from sklearn.ensemble import RandomForestRegressor
RandomForestRegressor(n_estimators=500, max_depth=None,
                      min_samples_leaf=5, n_jobs=-1).fit(X_tr,
y_tr)
```

Non-linear, robust, no scaling needed. Bias OOB-error available.

6 · Gradient boosting (THE workhorse)

```
import lightgbm as lgb
m = lgb.LGBMRegressor(
    n_estimators=2000, learning_rate=0.05,
    num_leaves=63, min_child_samples=20,
    subsample=0.8, colsample_bytree=0.8,
    reg_alpha=0.0, reg_lambda=1.0,
    n_jobs=-1, random_state=42)
m.fit(X_tr, y_tr,
      eval_set=[(X_va, y_va)],
      callbacks=[lgb.early_stopping(50)])
```

XGBoost + CatBoost are interchangeable; CatBoost handles categoricals natively.

7 · SVR / GP / NN (advanced)

- **SVR**: small-N, non-linear, slow O(N²)-O(N³)
- **GaussianProcess**: probabilistic, small-N
- **MLP / Tabular NN**: only when N > 100k or non-tabular

Model picker

Situation	Start with
Need interpretability	OLS / Ridge
Sparse high-dim	Lasso / ElasticNet
Tabular, non-linear, >1k rows	LightGBM
Small N, smooth	SVR (RBF) / GP
Counts / positives	Poisson / Gamma GLM
Quantiles (P10/P90)	quantile GBM

◆ Metrics

RMSE / MAE / MAPE

Metric	Formula	When
RMSE	√mean((y−ŷ)²)	default; penalises large errors
MAE	mean|y−ŷ|	robust to outliers
MAPE	mean|y−ŷ|/|y|	units = %; FAILS at y≈0
sMAPE	2·|y−ŷ|/(|y|+|ŷ|)	bounded MAPE alternative
R²	1 − SSE/SST	relative fit; can be negative
Pinball	quantile loss	asymmetric / quantile reg

Canonical use

```
from sklearn.metrics import (mean_squared_error,
mean_absolute_error,
                             mean_absolute_percentage_error,
r2_score)
rmse = np.sqrt(mean_squared_error(y, y_hat))
mae  = mean_absolute_error(y, y_hat)
mape = mean_absolute_percentage_error(y, y_hat)
r2   = r2_score(y, y_hat)
```

Picking the metric

- **Symmetric, no outliers** → RMSE
- **Symmetric, outliers** → MAE
- **Relative errors** → MAPE (units don't matter)
- **Asymmetric loss** → Pinball / Huber
- **Report R²** alongside; never as the only metric

Log-target trap: RMSE on log(y) is NOT in original units. Always inverse-transform predictions before metrics.

● Cross-validation

KFold (iid)

```
from sklearn.model_selection import KFold, cross_val_score
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X_tr, y_tr, cv=kf,
                         scoring="neg_root_mean_squared_error",
n_jobs=-1)
print(-scores.mean(), scores.std())
```

TimeSeriesSplit (temporal)

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5, test_size=30, gap=0)
# Expanding window by default; for rolling pass max_train_size
```

Never shuffle on temporal data. Train always precedes val in time.

GroupKFold (grouped data)

```
from sklearn.model_selection import GroupKFold
gkf = GroupKFold(n_splits=5)
# groups = customer_id, batch_id, anything that must not split
across folds
```

Nested CV (unbiased tuning)

```
outer = KFold(5, shuffle=True, random_state=42)
inner = KFold(3, shuffle=True, random_state=42)
gs = GridSearchCV(pipe, grid, cv=inner,
scoring="neg_root_mean_squared_error")
nested_score = cross_val_score(gs, X, y, cv=outer,

scoring="neg_root_mean_squared_error")
```

Use when reporting performance estimate AND tuning. ~5× slower.

▲ Hyperparameter tuning

GridSearchCV (small grids)

```
grid = {"model__alpha": [0.01, 0.1, 1, 10]}
gs = GridSearchCV(pipe, grid, cv=5,
scoring="neg_root_mean_squared_error",
                  n_jobs=-1, refit=True)
gs.fit(X_tr, y_tr)
print(gs.best_params_, -gs.best_score_)
```

Optuna TPE (≥3 hp)

```
import optuna
def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("lr", 1e-3, 0.3,
log=True),
        "num_leaves":    trial.suggest_int("leaves", 15, 255,
log=True),
        "min_child_samples": trial.suggest_int("mcs", 5, 100),
        "reg_lambda":    trial.suggest_float("rl", 1e-3, 10,
log=True),
    }
    model = lgb.LGBMRegressor(**params, n_estimators=2000)
    scores = cross_val_score(model, X_tr, y_tr, cv=5,

scoring="neg_root_mean_squared_error", n_jobs=-1)
    return -scores.mean()
study = optuna.create_study(direction="minimize",

sampler=optuna.samplers.TPESampler(seed=42),

pruner=optuna.pruners.MedianPruner())
study.optimize(objective, n_trials=100, n_jobs=1)
print(study.best_params, study.best_value)
```

GBM tuning priority order

1. **n_estimators + learning_rate** (with early stopping)
2. **num_leaves / max_depth** (capacity)
3. **min_child_samples** (overfitting guard)
4. **subsample / colsample_bytree** (variance reduction)
5. **reg_alpha / reg_lambda** (L1/L2)

◆ Residual diagnostics

Standard 4-plot

```
resid = y_te - y_hat
fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax[0,0].scatter(y_hat, resid);  ax[0,0].set_title("Resid vs ŷ")
ax[0,0].axhline(0, c='r')
ax[0,1].hist(resid, bins=30);    ax[0,1].set_title("Hist")
import scipy.stats as st
st.probplot(resid, dist="norm", plot=ax[1,0])  # Q-Q
ax[1,1].scatter(range(len(resid)), resid)       # vs index/time
ax[1,1].set_title("Resid vs order")
```

What to look for

Pattern	Cause	Fix
Funnel (resid ↑ with ŷ)	heteroskedastic	log target / WLS / GBM
U-shape vs ŷ	missing non-linearity	add interactions / GBM
Trend vs time	concept drift	add time feature / re-train cadence
Q-Q tail deviation	fat-tailed residuals	Huber / quantile loss
Autocorr (DW≠2)	serial dependence	TS model / add lags

Feature importance + SHAP

```
import shap
explainer = shap.TreeExplainer(model)
sv = explainer(X_tr.sample(500))
shap.plots.beeswarm(sv); shap.plots.bar(sv)
# Single prediction:
shap.plots.waterfall(sv[0])
```

Prediction intervals

```
from sklearn.ensemble import GradientBoostingRegressor
lo = GradientBoostingRegressor(loss="quantile",
alpha=0.1).fit(X_tr, y_tr)
hi = GradientBoostingRegressor(loss="quantile",
alpha=0.9).fit(X_tr, y_tr)
# Or LGBM: objective="quantile", alpha=0.1
# Or MAPIE for conformal prediction (distribution-free):
from mapie.regression import MapieRegressor
mapie = MapieRegressor(estimator=pipe, cv=5).fit(X_tr, y_tr)
y_pred, intervals = mapie.predict(X_te, alpha=0.10)
```

◆ Persistence + deployment

Joblib (canonical)

```
import joblib
joblib.dump(gs.best_estimator_, "model.joblib", compress=3)
# Load in prod (Python only):
pipe = joblib.load("model.joblib")
y_hat = pipe.predict(X_new)
# Safer alternative — sklearn-recommended since 1.3:
import skops.io as sio
sio.dump(gs.best_estimator_, "model.skops")   # explicit type
allowlist on load
```

Pickle/joblib are Python-version + sklearn-version locked AND can execute arbitrary code on load. Pin versions in pyproject.toml; prefer `skops` for shareable artifacts.

ONNX (cross-runtime)

```
from skl2onnx import to_onnx
onx = to_onnx(pipe, X_tr[:1].astype(np.float32))
with open("model.onnx", "wb") as f:
f.write(onx.SerializeToString())
# Serve with ONNXRuntime in C++/Rust/Java/JS
```

FastAPI service skeleton

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
class Req(BaseModel): age: float; income: float; region: str
pipe = joblib.load("model.joblib")
@app.post("/predict")
def predict(r: Req):
    X = pd.DataFrame([r.dict()])
    return {"y_hat": float(pipe.predict(X)[0])}
```

Monitoring (drift)

- **Input drift**: PSI / KS test on each feature vs train
- **Prediction drift**: distribution of ŷ over time
- **Performance drift**: rolling RMSE when y arrives (lag)
- **Re-train trigger**: PSI > 0.25 OR RMSE > baseline · 1.2

▲ Top traps (6-month memory)

- **Leakage via scaler-on-all-data** — fit scaler on X_train ONLY, transform val/test with fitted scaler. `Pipeline` enforces this.
- **Random split on temporal data** — always use `TimeSeriesSplit` when rows have a time stamp.
- **Touching test more than once** — every peek "spends" your held-out estimate. Final test = ONE call.
- **Optimising R² without residual checks** — high R² + funnel pattern = wrong loss / heteroskedastic.
- **Reporting RMSE on log(y) as original units** — inverse-transform BEFORE the metric call.
- **Polynomial features without regularisation** — explodes p in p>>n regime.
- **Trusting single-model feature importance** — different models disagree; use SHAP + permutation importance.
- **No baseline** — your model isn't "good" until it beats DummyRegressor(mean) and a single-feature OLS.

◆ Quick-reference (recall in 5 seconds)

Task	Canonical pattern
Split	`train_test_split(test_size=0.2, random_state=42)`
Preprocess	`Pipeline([prep, model])` + `ColumnTransformer`
Baseline	`DummyRegressor` + `LinearRegression`
CV	`KFold(5, shuffle=True, random_state=42)`
Tune (small)	`GridSearchCV(pipe, grid, cv=5, scoring="neg_root_mean_squared_error")`
Tune (large)	Optuna TPE, 100 trials, MedianPruner
Best model (tabular)	LightGBM + early stopping
Diagnostics	Resid vs ŷ, Q-Q, SHAP beeswarm
Intervals	Quantile GBM or MAPIE
Persist	`joblib.dump(best_estimator_)`